

Toy Robot World — Project Info

A small client–server software system: a client sends JSON commands, the server runs the world simulation and supports multiple players over sockets.

What this project is

A real software system where a user controls robots in a 2D world by typing commands. It is split into two programs:

- Client: takes user input and sends it to the server.
- Server: owns the world and robot states, executes commands, and sends results back.

The client and server talk using JSON messages over a network socket.

What we are trying to achieve

- Let a user run commands like **launch**, **forward 5**, **turn left**, **state**, etc.
- The client sends a JSON request to the server.
- The server updates the robot/world, then returns a JSON response.
- Support multiple connected clients at the same time.

Main concepts being taught (in simple English)

1. Client–Server architecture

- Client (**RobotClient**) is the user interface.
- Server (**RobotServer**, **ClientHandler**) is the “game engine” and the source of truth.

Why it matters: this is how many real apps work (games, chat apps, web systems).

2. Networking with sockets

- Client connects with a **Socket(host, port)**.
- Server listens using **ServerSocket(port)** and waits for connections using **accept()**.

Why it matters: this is real computer-to-computer communication.

3. Threads / concurrency

- The server creates a new Thread (running a **ClientHandler**) for each connected client.

Why it matters: the server can handle many players at once.

4. Object-Oriented Programming (OOP): modeling real things

Classes represent “real world” ideas:

- Robots (name, position, direction, shields, status)
- A world (size, obstacles, robots inside it)
- Positions, directions, server requests/responses

Why it matters: you learn to design code around objects and responsibilities.

5. Encapsulation (protecting data)

Instead of changing robot position everywhere, code uses methods like:

- `moveRobot(...), updatePosition(...), rotateRobot(...)`

Why it matters: reduces bugs and keeps the rules in one place.

6. Inheritance + abstraction

- `BaseRobot` is an abstract class that holds shared robot behaviour.
- Concrete robots (like `SimpleRobot`) implement/extend that base.

Why it matters: you can add new robot types without rewriting everything.

7. Interfaces (contracts)

- `Iworld` defines what a world must be able to do (add robot, move, look, etc.).
- `RobotWorld` is one implementation of that interface.

Why it matters: makes the code easier to extend and test.

8. Enums (fixed choices)

- `Directions` (NORTH, SOUTH, EAST, WEST)
- `OperationalMode` (NORMAL, REPAIR, RELOAD, DEAD)

Why it matters: avoids "string typos" and makes state predictable.

9. Command pattern (clean command logic)

- There is a base `Command` type.
- Each action is its own class: `ForwardCommand`, `TurnCommand`, `LaunchCommand`, etc.
- A factory method (like `Command.generate(req)`) picks the correct command class.

Why it matters: avoids one huge `if/else` block and keeps code organised.

10. Shared protocol (client/server agreement)

- Shared code (`Protocol`, request/response classes) defines the JSON structure.

Why it matters: both sides must agree on the message format (like an API contract).

11. Serialization / deserialization (object ↔ JSON)

- Serialize: Java object → JSON string
- Deserialize: JSON string → Java object

Why it matters: this is how most systems communicate (APIs and services).

12. Testing (JUnit)

Tests check important rules:

- world movement and collisions
- correct JSON formatting

- correct command selection / dispatch

Why it matters: proves your code works and stays working as you change it.

13. Build tools + dependencies (Maven)

`pom.xml` manages libraries like:

- Jackson (JSON)
- JUnit (tests)
- SLF4J / Logback (logging)

Why it matters: real projects use dependency management and repeatable builds.

14. Logging + observability

- Logs help you see what the server / client is doing.
- Prometheus metrics hooks exist for monitoring.

Why it matters: real systems need visibility to debug and measure behaviour.

15. Configuration files (.properties)

- `world.properties` is used to load / set up the world (like maps and settings).

Why it matters: avoids hard-coding everything in code.

One-sentence summary

This project teaches you how to build a multi-part program using OOP + patterns, where a client sends JSON commands to a server over sockets, the server runs the world simulation and supports multiple clients with threads, and you keep it reliable with tests, logging, and configuration.